

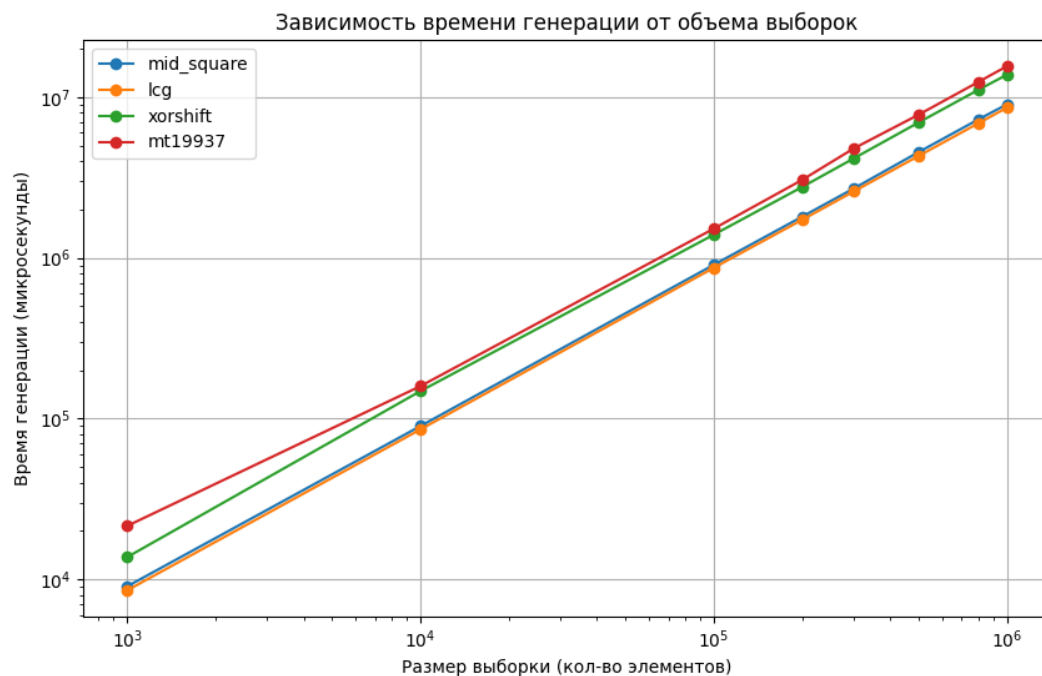
Отчет к лабораторной работе №3

Труфанова Анастасия СКБ222

Ссылка на исходный код программы в репозитории

https://github.com/neko-nyashka/lab3_generation/tree/develop/src

График зависимости времени генерации псевдослучайных чисел от объема выборки



My Project

Generated by Doxygen 1.9.5

1 File Index	1
1.1 File List	1
2 File Documentation	3
2.1 /Users/anastasiatrufanova/Desktop/lab3_generation/src/gen.cpp File Reference	3
2.1.1 Function Documentation	4
2.1.1.1 generate_and_analyze()	4
2.1.1.2 lcg_next()	4
2.1.1.3 lcg_sample()	4
2.1.1.4 main()	5
2.1.1.5 mid_square_next()	5
2.1.1.6 mid_square_sample()	5
2.1.1.7 mt_sample()	6
2.1.1.8 time()	6
2.1.1.9 time_seed()	6
2.1.1.10 xorshift_next()	6
2.1.1.11 xorshift_sample()	7
2.2 gen.cpp	7
2.3 /Users/anastasiatrufanova/Desktop/lab3_generation/src/gen.h File Reference	10
2.3.1 Function Documentation	10
2.3.1.1 generate_and_analyze()	10
2.3.1.2 lcg_next()	11
2.3.1.3 lcg_sample()	11
2.3.1.4 mid_square_next()	12
2.3.1.5 mid_square_sample()	12
2.3.1.6 mt_sample()	12
2.3.1.7 time()	13
2.3.1.8 time_seed()	13
2.3.1.9 xorshift_next()	13
2.3.1.10 xorshift_sample()	14
2.4 gen.h	14
2.5 /Users/anastasiatrufanova/Desktop/lab3_generation/src/nist.cpp File Reference	14
2.5.1 Function Documentation	15
2.5.1.1 block_frequency_test()	15
2.5.1.2 cusum_test()	15
2.5.1.3 longest_run_test()	16
2.5.1.4 monobit_test()	16
2.5.1.5 Phi()	16
2.5.1.6 runs_test()	17
2.6 nist.cpp	17
2.7 /Users/anastasiatrufanova/Desktop/lab3_generation/src/nist.h File Reference	19
2.7.1 Function Documentation	20

2.7.1.1 block_frequency_test()	20
2.7.1.2 cusum_test()	20
2.7.1.3 longest_run_test()	21
2.7.1.4 monobit_test()	21
2.7.1.5 Phi()	21
2.7.1.6 runs_test()	22
2.8 nist.h	22
2.9 /Users/anastasiatrufanova/Desktop/lab3_generation/src/stats.cpp File Reference	22
2.9.1 Function Documentation	23
2.9.1.1 checking_chi2()	23
2.9.1.2 chi2_exp()	23
2.9.1.3 chi2_teor()	24
2.9.1.4 cv()	24
2.9.1.5 mean()	25
2.9.1.6 stddev()	25
2.10 stats.cpp	25
2.11 /Users/anastasiatrufanova/Desktop/lab3_generation/src/stats.h File Reference	26
2.11.1 Function Documentation	27
2.11.1.1 checking_chi2()	27
2.11.1.2 chi2_exp()	27
2.11.1.3 chi2_teor()	28
2.11.1.4 cv()	28
2.11.1.5 mean()	29
2.11.1.6 stddev()	29
2.12 stats.h	29
Index	31

Chapter 1

File Index

1.1 File List

Here is a list of all files with brief descriptions:

/Users/anastasiatrufanova/Desktop/lab3_generation/src/ gen.cpp	3
/Users/anastasiatrufanova/Desktop/lab3_generation/src/ gen.h	10
/Users/anastasiatrufanova/Desktop/lab3_generation/src/ nist.cpp	14
/Users/anastasiatrufanova/Desktop/lab3_generation/src/ nist.h	19
/Users/anastasiatrufanova/Desktop/lab3_generation/src/ stats.cpp	22
/Users/anastasiatrufanova/Desktop/lab3_generation/src/ stats.h	26

Chapter 2

File Documentation

2.1 /Users/anastasiatrufanova/Desktop/lab3_generation/src/gen.cpp File Reference

```
#include "gen.h"  
#include "nist.h"  
#include "stats.h"
```

Functions

- `uint32_t time_seed ()`
Returns the current time in microseconds as a 32-bit integer.
- `uint32_t mid_square_next (uint32_t x)`
Computes the next value using the mid-square method.
- `void mid_square_sample (uint32_t seed, int len, uint32_t *a)`
Generates a sequence using the mid-square method.
- `uint32_t lcg_next (uint32_t x)`
Computes the next value using the Linear Congruential Generator (LCG) method.
- `void lcg_sample (uint32_t seed, int len, uint32_t *a)`
Generates a sequence using the LCG method.
- `uint32_t xorshift_next (uint32_t x)`
Computes the next value using the xorshift method.
- `void xorshift_sample (uint32_t seed, int len, uint32_t *a)`
Generates a sequence using the xorshift method.
- `void mt_sample (uint32_t seed, int len, uint32_t *a)`
Generates a sequence using the Mersenne Twister (mt19937) generator.
- `void generate_and_analyze (std::vector< std::vector< uint32_t > > &samples, void(*generator)(uint32_t, int, uint32_t *), const std::string &name)`
Generates multiple samples using the provided generator and performs statistical analysis.
- `void time ()`
Benchmarks the execution time of different random number generators and logs results.
- `int main ()`

2.1.1 Function Documentation

2.1.1.1 `generate_and_analyze()`

```
void generate_and_analyze (
    std::vector< std::vector< uint32_t > > & samples,
    void(*) (uint32_t, int, uint32_t *) generator,
    const std::string & name )
```

Generates multiple samples using the provided generator and performs statistical analysis.

Parameters

<i>samples</i>	Output vector of sequences.
<i>generator</i>	Pointer to a generator function.
<i>name</i>	Name of the generator (used for logging/output).

Definition at line 77 of file [gen.cpp](#).

2.1.1.2 `lcg_next()`

```
uint32_t lcg_next (
    uint32_t x )
```

Computes the next value using the Linear Congruential Generator (LCG) method.

Parameters

<i>x</i>	The previous value.
----------	---------------------

Returns

The next pseudo-random number.

Definition at line 31 of file [gen.cpp](#).

2.1.1.3 `lcg_sample()`

```
void lcg_sample (
    uint32_t seed,
    int len,
    uint32_t * a )
```

Generates a sequence using the LCG method.

Parameters

<i>seed</i>	Initial seed value.
<i>len</i>	Length of the sequence.
<i>a</i>	Output array for the generated numbers.

Definition at line 36 of file [gen.cpp](#).

2.1.1.4 main()

```
int main ( )
```

Definition at line 154 of file [gen.cpp](#).

2.1.1.5 mid_square_next()

```
uint32_t mid_square_next (
    uint32_t x )
```

Computes the next value using the mid-square method.

Parameters

<i>x</i>	The previous value.
----------	---------------------

Returns

The next pseudo-random number.

Definition at line 12 of file [gen.cpp](#).

2.1.1.6 mid_square_sample()

```
void mid_square_sample (
    uint32_t seed,
    int len,
    uint32_t * a )
```

Generates a sequence using the mid-square method.

Parameters

<i>seed</i>	Initial seed value.
<i>len</i>	Length of the sequence.
<i>a</i>	Output array for the generated numbers.

Definition at line 19 of file [gen.cpp](#).

2.1.1.7 mt_sample()

```
void mt_sample (
    uint32_t seed,
    int len,
    uint32_t * a )
```

Generates a sequence using the Mersenne Twister (mt19937) generator.

Parameters

<i>seed</i>	Initial seed value.
<i>len</i>	Length of the sequence.
<i>a</i>	Output array for the generated numbers.

Definition at line 68 of file [gen.cpp](#).

2.1.1.8 time()

```
void time ( )
```

Benchmarks the execution time of different random number generators and logs results.

Definition at line 105 of file [gen.cpp](#).

2.1.1.9 time_seed()

```
uint32_t time_seed ( )
```

Returns the current time in microseconds as a 32-bit integer.

Returns

A time-based seed value.

Definition at line 5 of file [gen.cpp](#).

2.1.1.10 xorshift_next()

```
uint32_t xorshift_next (
    uint32_t x )
```

Computes the next value using the xorshift method.

Parameters

<i>x</i>	The previous value.
----------	---------------------

Returns

The next pseudo-random number.

Definition at line 48 of file [gen.cpp](#).

2.1.1.11 xorshift_sample()

```
void xorshift_sample (
    uint32_t seed,
    int len,
    uint32_t * a )
```

Generates a sequence using the xorshift method.

Parameters

<i>seed</i>	Initial seed value.
<i>len</i>	Length of the sequence.
<i>a</i>	Output array for the generated numbers.

Definition at line 56 of file [gen.cpp](#).

2.2 gen.cpp

[Go to the documentation of this file.](#)

```
00001 #include "gen.h"
00002 #include "nist.h"
00003 #include "stats.h"
00004
00005 uint32_t time_seed() {
00006     auto now = std::chrono::high_resolution_clock::now();
00007     auto micro = std::chrono::time_point_cast<std::chrono::microseconds>(now);
00008     return uint32_t(micro.time_since_epoch().count());
00009 }
00010
00011 // The middle squares method
00012 uint32_t mid_square_next(uint32_t x) {
00013     uint64_t tmp = uint64_t(x) * x;
00014     uint64_t mid = tmp & 0xFFFFFFFF0000;
00015     return uint32_t(mid >> 16);
00016 }
00017
00018
00019 void mid_square_sample(uint32_t seed, int len, uint32_t* a) {
00020     uint32_t x = seed;
00021     for (int i = 0; i < len; i++) {
00022         uint32_t next = mid_square_next(x);
00023         next = next == 0 ? seed + i : next;
00024         a[i] = next;
00025         x = next;
00026     }
```

```

00027 }
00028
00029
00030 // Linear congruent method  $r_{i+1} = (k * r_i + b) \% M$ 
00031 uint32_t lcg_next(uint32_t x) {
00032     uint64_t product = uint64_t(1664525) * x + 1013904223u;
00033     return uint32_t(product);
00034 }
00035
00036 void lcg_sample(uint32_t seed, int len, uint32_t* a) {
00037     uint32_t x = seed;
00038
00039     for (int i = 0; i < len; i++) {
00040         uint32_t next = lcg_next(x);
00041         a[i] = next;
00042         x = next;
00043     }
00044 }
00045
00046
00047 // Xorshift
00048 uint32_t xorshift_next(uint32_t x) {
00049     x ^= x << 13;
00050     x ^= x >> 17;
00051     x ^= x << 5;
00052     return x;
00053 }
00054
00055
00056 void xorshift_sample(uint32_t seed, int len, uint32_t* a) {
00057     uint32_t x = seed;
00058
00059     for (int i = 0; i < len; i++) {
00060         uint32_t next = xorshift_next(x);
00061         a[i] = next;
00062         x = next;
00063     }
00064 }
00065
00066
00067
00068 void mt_sample(uint32_t seed, int len, uint32_t* a) {
00069     std::mt19937 gen(seed);
00070     for (int i = 0; i < len; i++) {
00071         a[i] = gen();
00072     }
00073 }
00074
00075
00076
00077 void generate_and_analyze(
00078     std::vector<std::vector<uint32_t>& samples, void (*generator)(uint32_t, int, uint32_t*), const
00079     std::string& name) {
00080     int sample_count = 20;
00081     int sample_size = 1000;
00082     std::cout << "\n" << name << "\n";
00083     samples.clear();
00084     samples.reserve(sample_count);
00085
00086     for (int i = 1; i <= sample_count; i++) {
00087         std::vector<uint32_t> sample(sample_size);
00088         generator(time_seed() + i*i, sample_size, sample.data());
00089         samples.push_back(sample);
00090
00091         long double m = mean(sample.data(), sample_size);
00092         long double sd = stddev(sample.data(), sample_size, m);
00093         long double c = cv(m, sd);
00094
00095         std::cout << "Sample " << i
00096                 << ": size = " << sample_size
00097                 << ", mean: " << uint64_t(m)
00098                 << ", stddev: " << uint64_t(sd)
00099                 << ", cv: " << c << "\n";
00100     }
00101 }
00102
00103
00104
00105 void time() {
00106     std::ofstream fout("results.csv");
00107     int sizes[13] = {1000, 2000, 3000, 10000, 20000, 30000, 50000, 100000, 200000, 300000, 500000,
00108                     800000, 1000000};
00109     if (!fout) {
00110         std::cerr << "Error opening file" << "\n";
00111         return;
00112     }

```

```

00112
00113     fout << "size,mid_square,lcg,xorshift,mt19937\n";
00114     int max_size = 1000000;
00115     std::vector<uint32_t> buffer(max_size);
00116     for (int len : sizes) {
00117         uint32_t seed = time_seed();
00118
00119         // mid_square_sample
00120         auto start = std::chrono::high_resolution_clock::now();
00121         mid_square_sample(seed, len, buffer.data());
00122         auto end = std::chrono::high_resolution_clock::now();
00123         double mid_square_time = std::chrono::duration<double, std::milli>(end - start).count();
00124
00125         // lcg_sample
00126         start = std::chrono::high_resolution_clock::now();
00127         lcg_sample(seed, len, buffer.data());
00128         end = std::chrono::high_resolution_clock::now();
00129         double lcg_time = std::chrono::duration<double, std::milli>(end - start).count();
00130
00131         // xorshift_sample
00132         start = std::chrono::high_resolution_clock::now();
00133         xorshift_sample(seed, len, buffer.data());
00134         end = std::chrono::high_resolution_clock::now();
00135         double xorshift_time = std::chrono::duration<double, std::milli>(end - start).count();
00136
00137         // mt_sample
00138         start = std::chrono::high_resolution_clock::now();
00139         mt_sample(seed, len, buffer.data());
00140         end = std::chrono::high_resolution_clock::now();
00141         double mt_time = std::chrono::duration<double, std::milli>(end - start).count();
00142
00143         fout << len << ", "
00144             << mid_square_time << ", "
00145             << lcg_time << ", "
00146             << xorshift_time << ", "
00147             << mt_time << "\n";
00148     }
00149 }
00150
00151     fout.close();
00152 }
00153
00154 int main() {
00155
00156     std::vector<std::vector<uint32_t> mid_square_samples;
00157     std::vector<std::vector<uint32_t> lcg_samples;
00158     std::vector<std::vector<uint32_t> xorshift_samples;
00159     std::vector<std::vector<uint32_t> mt_samples;
00160
00161     generate_and_analyze(mid_square_samples, mid_square_sample, "The middle squares method");
00162     generate_and_analyze(lcg_samples, lcg_sample, "Linear congruent method");
00163     generate_and_analyze(xorshift_samples, xorshift_sample, "Xorshift method");
00164     generate_and_analyze(mt_samples, mt_sample, "MT19937 method");
00165
00166     std::vector<std::vector<std::vector<uint32_t>> all_samples = {
00167         mid_square_samples,
00168         lcg_samples,
00169         xorshift_samples,
00170         mt_samples
00171     };
00172
00173     std::vector<std::string> method_names = {
00174         "mid_square", "lcg", "xorshift", "mt19937"
00175     };
00176
00177     for (int m = 0; m < int(all_samples.size()); m++) {
00178         auto& samples = all_samples[m];
00179         std::string method = method_names[m];
00180         std::cout << '\n' << method << '\n';
00181         for (size_t i = 0; i < samples.size(); i++) {
00182             auto& sample = samples[i];
00183             std::cout << "| " << std::setw(2) << i << " ";
00184             checking_chi2(sample);
00185             monobit_test(sample);
00186             runs_test(sample);
00187             cusum_test(sample);
00188             block_frequency_test(sample);
00189             longest_run_test(sample);
00190             std::cout << '\n';
00191         }
00192     }
00193
00194     time();
00195
00196
00197
00198     return 0;

```



```
00199 }
```

2.3 /Users/anastasiatrufanova/Desktop/lab3_generation/src/gen.h File Reference

```
#include <chrono>
#include <random>
#include <cmath>
#include <iostream>
#include <fstream>
#include <vector>
```

Functions

- `uint32_t time_seed ()`
Returns the current time in microseconds as a 32-bit integer.
- `uint32_t mid_square_next (uint32_t x)`
Computes the next value using the mid-square method.
- `void mid_square_sample (uint32_t seed, int len, uint32_t *a)`
Generates a sequence using the mid-square method.
- `uint32_t lcg_next (uint32_t x)`
Computes the next value using the Linear Congruential Generator (LCG) method.
- `void lcg_sample (uint32_t seed, int len, uint32_t *a)`
Generates a sequence using the LCG method.
- `uint32_t xorshift_next (uint32_t x)`
Computes the next value using the xorshift method.
- `void xorshift_sample (uint32_t seed, int len, uint32_t *a)`
Generates a sequence using the xorshift method.
- `void mt_sample (uint32_t seed, int len, uint32_t *a)`
Generates a sequence using the Mersenne Twister (mt19937) generator.
- `void generate_and_analyze (std::vector< std::vector< uint32_t > > &samples, void(*generator)(uint32_t, int, uint32_t *), const std::string &name)`
Generates multiple samples using the provided generator and performs statistical analysis.
- `void time ()`
Benchmarks the execution time of different random number generators and logs results.

2.3.1 Function Documentation

2.3.1.1 generate_and_analyze()

```
void generate_and_analyze (
    std::vector< std::vector< uint32_t > > & samples,
    void(*) (uint32_t, int, uint32_t *) generator,
    const std::string & name )
```

Generates multiple samples using the provided generator and performs statistical analysis.

Parameters

<i>samples</i>	Output vector of sequences.
<i>generator</i>	Pointer to a generator function.
<i>name</i>	Name of the generator (used for logging/output).

Definition at line 77 of file [gen.cpp](#).

2.3.1.2 lcg_next()

```
uint32_t lcg_next (  
    uint32_t x )
```

Computes the next value using the Linear Congruential Generator (LCG) method.

Parameters

<i>x</i>	The previous value.
----------	---------------------

Returns

The next pseudo-random number.

Definition at line 31 of file [gen.cpp](#).

2.3.1.3 lcg_sample()

```
void lcg_sample (  
    uint32_t seed,  
    int len,  
    uint32_t * a )
```

Generates a sequence using the LCG method.

Parameters

<i>seed</i>	Initial seed value.
<i>len</i>	Length of the sequence.
<i>a</i>	Output array for the generated numbers.

Definition at line 36 of file [gen.cpp](#).

2.3.1.4 mid_square_next()

```
uint32_t mid_square_next (
    uint32_t x )
```

Computes the next value using the mid-square method.

Parameters

<i>x</i>	The previous value.
----------	---------------------

Returns

The next pseudo-random number.

Definition at line 12 of file [gen.cpp](#).

2.3.1.5 mid_square_sample()

```
void mid_square_sample (
    uint32_t seed,
    int len,
    uint32_t * a )
```

Generates a sequence using the mid-square method.

Parameters

<i>seed</i>	Initial seed value.
<i>len</i>	Length of the sequence.
<i>a</i>	Output array for the generated numbers.

Definition at line 19 of file [gen.cpp](#).

2.3.1.6 mt_sample()

```
void mt_sample (
    uint32_t seed,
    int len,
    uint32_t * a )
```

Generates a sequence using the Mersenne Twister (mt19937) generator.

Parameters

<i>seed</i>	Initial seed value.
<i>len</i>	Length of the sequence.
<i>a</i>	Output array for the generated numbers.

Definition at line 68 of file [gen.cpp](#).

2.3.1.7 time()

```
void time ( )
```

Benchmarks the execution time of different random number generators and logs results.

Definition at line 105 of file [gen.cpp](#).

2.3.1.8 time_seed()

```
uint32_t time_seed ( )
```

Returns the current time in microseconds as a 32-bit integer.

Returns

A time-based seed value.

Definition at line 5 of file [gen.cpp](#).

2.3.1.9 xorshift_next()

```
uint32_t xorshift_next (
    uint32_t x )
```

Computes the next value using the xorshift method.

Parameters

<i>x</i>	The previous value.
----------	---------------------

Returns

The next pseudo-random number.

Definition at line 48 of file [gen.cpp](#).

2.3.1.10 xorshift_sample()

```
void xorshift_sample (
    uint32_t seed,
    int len,
    uint32_t * a )
```

Generates a sequence using the xorshift method.

Parameters

<i>seed</i>	Initial seed value.
<i>len</i>	Length of the sequence.
<i>a</i>	Output array for the generated numbers.

Definition at line 56 of file [gen.cpp](#).

2.4 gen.h

[Go to the documentation of this file.](#)

```
00001 #ifndef GEN_H
00002 #define GEN_H
00003
00004 #include <chrono>
00005 #include <random>
00006 #include <cmath>
00007 #include <iostream>
00008 #include <fstream>
00009 #include <vector>
00010
00015 uint32_t time_seed();
00016
00022 uint32_t mid_square_next(uint32_t x);
00023
00030 void mid_square_sample(uint32_t seed, int len, uint32_t* a);
00031
00037 uint32_t lcg_next(uint32_t x);
00038
00045 void lcg_sample(uint32_t seed, int len, uint32_t* a);
00046
00052 uint32_t xorshift_next(uint32_t x);
00053
00060 void xorshift_sample(uint32_t seed, int len, uint32_t* a);
00061
00068 void mt_sample(uint32_t seed, int len, uint32_t* a);
00069
00070
00077 void generate_and_analyze(std::vector<std::vector<uint32_t>>& samples,
00078                             void (*generator)(uint32_t, int, uint32_t*),
00079                             const std::string& name);
00080
00084 void time();
00085
00086
00087 #endif
```

2.5 /Users/anastasiatrufanova/Desktop/lab3_generation/src/nist.cpp File Reference

```
#include "nist.h"
```

Functions

- int [monobit_test](#) (const std::vector< uint32_t > &sample)
Performs the monobit test (frequency test) for randomness.
- int [runs_test](#) (const std::vector< uint32_t > &sample)
Performs the runs test for randomness.
- double [Phi](#) (double x)
Computes the cumulative distribution function (CDF) of the standard normal distribution.
- int [cusum_test](#) (const std::vector< uint32_t > &sample)
Performs the cumulative sums (Cusum) test for randomness.
- int [block_frequency_test](#) (const std::vector< uint32_t > &sample)
Performs the block frequency test.
- int [longest_run_test](#) (const std::vector< uint32_t > &sample)
Performs the longest run of ones in a block test.

2.5.1 Function Documentation

2.5.1.1 [block_frequency_test\(\)](#)

```
int block_frequency_test (
    const std::vector< uint32_t > & sample )
```

Performs the block frequency test.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 94 of file [nist.cpp](#).

2.5.1.2 [cusum_test\(\)](#)

```
int cusum_test (
    const std::vector< uint32_t > & sample )
```

Performs the cumulative sums (Cusum) test for randomness.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 64 of file [nist.cpp](#).

2.5.1.3 longest_run_test()

```
int longest_run_test (
    const std::vector< uint32_t > & sample )
```

Performs the longest run of ones in a block test.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 127 of file [nist.cpp](#).

2.5.1.4 monobit_test()

```
int monobit_test (
    const std::vector< uint32_t > & sample )
```

Performs the monobit test (frequency test) for randomness.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 5 of file [nist.cpp](#).

2.5.1.5 Phi()

```
double Phi (
    double x )
```

Computes the cumulative distribution function (CDF) of the standard normal distribution.

Parameters

<i>x</i>	Input value.
----------	--------------

Returns

The CDF value (Φ).

Definition at line 59 of file [nist.cpp](#).

2.5.1.6 runs_test()

```
int runs_test (
    const std::vector< uint32_t > & sample )
```

Performs the runs test for randomness.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 28 of file [nist.cpp](#).

2.6 nist.cpp

[Go to the documentation of this file.](#)

```
00001 #include "nist.h"
00002
00003 // Frequency (Monobit) Test
00004 // To check the balance between zeros and ones in the entire sequence.
00005 int monobit_test(const std::vector<uint32_t>& sample) {
00006     int n = (sample.size() * 32);
00007     int sum = 0;
00008     for (uint32_t num : sample) {
00009         for (int i = 0; i < 32; ++i) {
00010             int bit = (num >> i) & 1;
00011             sum += (bit == 1) ? 1 : -1;
00012         }
00013     }
00014     double s_obs = fabs(sum) / sqrt(n);
00015     double p_val = erfc(s_obs / M_SQRT2);
00016     if (p_val > 0.01) {
00017         std::cout<<"| Monobit PASS ";
00018     } else {
00019         std::cout<<"| Monobit FAIL ";
00020     }
00021     return p_val > 0.01;
00022 }
00023
00024
00025
00026 // Runs Test
```



```

00027 // To check the alternation of zeros and ones.
00028 int runs_test(const std::vector<uint32_t>& sample) {
00029     int n = (sample.size() * 32);
00030     int one_count = 0;
00031     int runs = 1;
00032     int pred_bit = 0, bit = 0;
00033     for (uint32_t num : sample) {
00034         for (int i = 0; i < 32; ++i) {
00035             if (i > 0) pred_bit = bit;
00036             bit = (num >> i) & 1;
00037             if (bit != pred_bit) runs++;
00038             one_count += bit;
00039         }
00040     }
00041     double pi = double(one_count) / n;
00042     double p_val;
00043     if (fabs(pi - 0.5) >= 2.0 / sqrt(n)) {
00044         p_val = 0.0;
00045     } else {
00046         double numerator = runs - 2.0 * n * pi * (1 - pi);
00047         double denominator = 2.0 * sqrt(n) * pi * (1 - pi);
00048         p_val = erfc(fabs(numerator) / denominator);
00049     }
00050     if (p_val > 0.01) {
00051         std::cout<<"| Runs PASS ";
00052     } else {
00053         std::cout<<"| Runs FAIL ";
00054     }
00055     return p_val > 0.01;
00056 }
00057
00058 // Standard Normal Distribution Function
00059 double Phi(double x) {
00060     return 0.5 * (1.0 + std::erf(x / std::sqrt(2.0)));
00061 }
00062 // Cumulative Sums (Cusums) Test
00063 // To check the maximum deviation of the accumulated amount.
00064 int cusum_test(const std::vector<uint32_t>& sample) {
00065     int n = (sample.size() * 32);
00066     int sum = 0, max_sum = 0;
00067     for (uint32_t num : sample) {
00068         for (int i = 0; i < 32; ++i) {
00069             int bit = (num >> i) & 1;
00070             sum += (bit == 1) ? 1 : -1;
00071             if (abs(sum) > max_sum) max_sum = abs(sum);
00072         }
00073     }
00074
00075     double z = max_sum;
00076     double p = 0.0;
00077     int k_start = int((-n/z + 1)/4);
00078     int k_end = int((n/z - 1)/4);
00079
00080     for (int k = k_start; k <= k_end; k++) {
00081         p += Phi((4*k+1)*z/sqrt(n)) - Phi((4*k-1)*z/sqrt(n));
00082     }
00083     double p_val = 1.0 - p;
00084     if (p_val > 0.01) {
00085         std::cout<<"| CuSum PASS ";
00086     } else {
00087         std::cout<<"| Cusum FAIL ";
00088     }
00089     return p_val > 0.01;
00090 }
00091
00092 // Block Frequency Test
00093 // Check the uniformity of small blocks.
00094 int block_frequency_test(const std::vector<uint32_t>& sample) {
00095     int block_size = 128;
00096     int num_blocks = sample.size() * 32 / block_size;
00097     double chi_sq = 0.0;
00098     for (int i = 0; i < num_blocks; i++) {
00099         int ones = 0;
00100         for (int ind = i * 4; ind < (i + 1) * 4; ++ind) {
00101             for (int j = 0; j < 32; ++j) {
00102                 ones += (sample[ind] >> j) & 1;
00103             }
00104         }
00105         double pi = double(ones) / block_size;
00106         chi_sq += (pi - 0.5) * (pi - 0.5);
00107     }
00108     chi_sq *= 4 * block_size;
00109
00110     // X -> N(v, 2v):
00111     // v = N_blocks; Z = (X - v) / sqrt(2v); p = erfc(|Z|/sqrt(2)).
00112     double nu = double(num_blocks);

```

```

00114     double Z = (chi_sq - nu) / sqrt(2.0 * nu);
00115     double p_val = erfc(abs(Z) / sqrt(2.0));
00116     if (p_val > 0.01) {
00117         std::cout<<"| BlockFreq PASS ";
00118     } else {
00119         std::cout<<"| BlockFreq FAIL ";
00120     }
00121     return p_val > 0.01;
00122 }
00123
00124
00125 // Longest Run of Ones in a Block (Longest series Test)
00126 // To check the length of the longest sequence of units.
00127 int longest_run_test(const std::vector<uint32_t>& sample) {
00128     int total_bits = sample.size() * 32;
00129     int M = 128;
00130     int K = 3;
00131
00132     std::vector<double> pi = { 0.1174, 0.2430, 0.2493, 0.1752, 0.1027, 0.1124 };
00133
00134     int num_blocks = total_bits / M;
00135     std::vector<int> counts(K + 1, 0);
00136     int bit_pos = 0, run = 0, max_run = 0;
00137
00138     for (uint32_t x : sample) {
00139         for (int b = 0; b < 32; ++b) {
00140             int bit = (x >> b) & 1;
00141             if (bit) {
00142                 run++;
00143                 if (run > max_run) max_run = run;
00144             } else {
00145                 run = 0;
00146             }
00147             bit_pos++;
00148             if (bit_pos == M) {
00149                 int idx = 0;
00150                 if (max_run <= 4) idx = 0;
00151                 else if (max_run == 5) idx = 1;
00152                 else if (max_run == 6) idx = 2;
00153                 else if (max_run == 7) idx = 3;
00154                 else if (max_run == 8) idx = 4;
00155                 else idx = 5;
00156
00157                 counts[idx]++;
00158                 bit_pos = 0;
00159                 run = 0;
00160                 max_run = 0;
00161             }
00162         }
00163     }
00164
00165     double chi2 = 0.0;
00166     for (int i = 0; i <= K; ++i) {
00167         double expected = double(num_blocks) * pi[i];
00168         double diff = double(counts[i]) - expected;
00169         chi2 += (diff * diff) / expected;
00170     }
00171
00172     double Z = (chi2 - K) / sqrt(2.0 * K);
00173     double p_val = erfc(fabs(Z) / sqrt(2.0));
00174
00175     if (p_val > 0.01) {
00176         std::cout << "| LRO PASS ";
00177     } else {
00178         std::cout << "| LRO FAIL ";
00179     }
00180
00181     return p_val > 0.01;
00182 }

```

2.7 /Users/anastasiatrufanova/Desktop/lab3_generation/src/nist.h File Reference

```
#include <iostream>
```

Functions

- int [monobit_test](#) (const std::vector< uint32_t > &sample)

- Performs the monobit test (frequency test) for randomness.*
- int [runs_test](#) (const std::vector< uint32_t > &sample)
Performs the runs test for randomness.
- int [cusum_test](#) (const std::vector< uint32_t > &sample)
Performs the cumulative sums (Cusum) test for randomness.
- int [block_frequency_test](#) (const std::vector< uint32_t > &sample)
Performs the block frequency test.
- int [longest_run_test](#) (const std::vector< uint32_t > &sample)
Performs the longest run of ones in a block test.
- double [Phi](#) (double x)
Computes the cumulative distribution function (CDF) of the standard normal distribution.

2.7.1 Function Documentation

2.7.1.1 [block_frequency_test\(\)](#)

```
int block_frequency_test (
    const std::vector< uint32_t > & sample )
```

Performs the block frequency test.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 94 of file [nist.cpp](#).

2.7.1.2 [cusum_test\(\)](#)

```
int cusum_test (
    const std::vector< uint32_t > & sample )
```

Performs the cumulative sums (Cusum) test for randomness.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 64 of file [nist.cpp](#).

2.7.1.3 longest_run_test()

```
int longest_run_test (
    const std::vector< uint32_t > & sample )
```

Performs the longest run of ones in a block test.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 127 of file [nist.cpp](#).

2.7.1.4 monobit_test()

```
int monobit_test (
    const std::vector< uint32_t > & sample )
```

Performs the monobit test (frequency test) for randomness.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 5 of file [nist.cpp](#).

2.7.1.5 Phi()

```
double Phi (
    double x )
```

Computes the cumulative distribution function (CDF) of the standard normal distribution.

Parameters

<i>x</i>	Input value.
----------	--------------

Returns

The CDF value (Φ).

Definition at line 59 of file [nist.cpp](#).

2.7.1.6 runs_test()

```
int runs_test (
    const std::vector< uint32_t > & sample )
```

Performs the runs test for randomness.

Parameters

<i>sample</i>	The sequence to test.
---------------	-----------------------

Returns

1 if the test is passed, otherwise 0.

Definition at line 28 of file [nist.cpp](#).

2.8 nist.h

[Go to the documentation of this file.](#)

```
00001 #ifndef NIST_H
00002 #define NIST_H
00003 #include <iostream>
00009 int monobit_test(const std::vector<uint32_t>& sample);
00010
00016 int runs_test(const std::vector<uint32_t>& sample);
00017
00023 int cusum_test(const std::vector<uint32_t>& sample);
00024
00030 int block_frequency_test(const std::vector<uint32_t>& sample);
00031
00037 int longest_run_test(const std::vector<uint32_t>& sample);
00038
00044 double Phi(double x);
00045
00046 #endif
```

2.9 /Users/anastasiatrufanova/Desktop/lab3_generation/src/stats.cpp
File Reference

```
#include "stats.h"
```

Functions

- long double [mean](#) (const uint32_t *data, int len)
Computes the mean of the given data.
- long double [stddev](#) (const uint32_t *data, int len, long double [mean](#))
Computes the standard deviation of the given data.
- long double [cv](#) (long double [mean](#), long double std_dev)
Computes the coefficient of variation.
- double [chi2_teor](#) (int v, double x_p)
Returns theoretical chi-squared value.
- double [chi2_exp](#) (const std::vector< uint32_t > &sample)
Calculates experimental chi-squared value.
- int [checking_chi2](#) (std::vector< uint32_t > &sample)
Checks the chi-squared hypothesis test for the sample.

2.9.1 Function Documentation

2.9.1.1 [checking_chi2\(\)](#)

```
int checking_chi2 (
    std::vector< uint32_t > & sample )
```

Checks the chi-squared hypothesis test for the sample.

Parameters

<i>sample</i>	Vector of integer sample data.
---------------	--------------------------------

Returns

0 if accepted, 1 if rejected.

Definition at line 60 of file [stats.cpp](#).

2.9.1.2 [chi2_exp\(\)](#)

```
double chi2_exp (
    const std::vector< uint32_t > & sample )
```

Calculates experimental chi-squared value.

Parameters

<i>sample</i>	Vector of integer sample data.
---------------	--------------------------------

Returns

Experimental chi-squared value.

Definition at line 29 of file [stats.cpp](#).

2.9.1.3 chi2_teor()

```
double chi2_teor (
    int v,
    double x_p )
```

Returns theoretical chi-squared value.

Parameters

ν	Degrees of freedom.
χ^2_{1-p}	Significance level (quantile).

Returns

Theoretical chi-squared value.

Definition at line 24 of file [stats.cpp](#).

2.9.1.4 cv()

```
long double cv (
    long double mean,
    long double std_dev )
```

Computes the coefficient of variation.

Parameters

<i>mean</i>	The mean of the data.
<i>std_dev</i>	The standard deviation.

Returns

The coefficient of variation (as a percentage).

Definition at line 18 of file [stats.cpp](#).

2.9.1.5 mean()

```
long double mean (
    const uint32_t * data,
    int len )
```

Computes the mean of the given data.

Parameters

<i>data</i>	Pointer to the array of values.
<i>len</i>	Number of elements in the array.

Returns

The mean value.

Definition at line 2 of file [stats.cpp](#).

2.9.1.6 stddev()

```
long double stddev (
    const uint32_t * data,
    int len,
    long double mean )
```

Computes the standard deviation of the given data.

Parameters

<i>data</i>	Pointer to the array of values.
<i>len</i>	Number of elements in the array.
<i>mean</i>	The mean of the data.

Returns

The standard deviation.

Definition at line 9 of file [stats.cpp](#).

2.10 stats.cpp

[Go to the documentation of this file.](#)

```
00001 #include "stats.h"
00002 long double mean(const uint32_t* data, int len) {
00003     uint64_t sum = 0;
00004     for (int i = 0; i < len; i++) {
00005         sum += data[i];
```



```

00006     }
00007     return sum / len;
00008 }
00009 long double stddev(const uint32_t* data, int len, long double mean) {
00010     long double sq_sum = 0;
00011
00012     for (int i = 0; i < len; i++) {
00013         sq_sum += (data[i] - mean) * (data[i] - mean);
00014     }
00015     return sqrtl(sq_sum / (len - 1));
00016 }
00017 }
00018 long double cv(long double mean, long double std_dev) {
00019     return (std_dev / mean) * 100;
00020 }
00021
00022
00023
00024 double chi2_teor(int v, double x_p) {
00025     return v + sqrt(2.0 * v) * x_p + (2.0/3.0) * (x_p * x_p) - (2.0/3.0);
00026 }
00027
00028
00029 double chi2_exp(const std::vector<uint32_t>& sample) {
00030
00031     int bins = 1+ std::floor(log2(sample.size()));
00032
00033     const uint64_t total_range = static_cast<uint64_t>(UINT32_MAX) + 1;
00034     const uint64_t bin_size = total_range / bins;
00035
00036     std::vector<int> freq(bins, 0);
00037
00038
00039     for (uint32_t num : sample) {
00040         uint64_t value = num;
00041         int bin_index = value / bin_size;
00042         if (bin_index >= bins) bin_index = bins - 1;
00043         freq[bin_index]++;
00044     }
00045
00046     double n = sample.size();
00047     double expected = n / bins;
00048
00049
00050     double chi_square = 0.0;
00051     for (int count : freq) {
00052         double diff = count - expected;
00053         chi_square += (diff * diff) / expected;
00054     }
00055
00056     return chi_square;
00057 }
00058
00059
00060 int checking_chi2(std::vector<uint32_t> &sample) {
00061     int n = sample.size();
00062     int v = floor(log2(double(n)));
00063     double x_25 = -0.674;
00064     double x_75 = 0.674;
00065
00066     double chi2_e = chi2_exp(sample);
00067     double chi2_teor_left = chi2_teor(v, x_25);
00068     double chi2_teor_right = chi2_teor(v, x_75);
00069     int passed = chi2_teor_left < chi2_e && chi2_e < chi2_teor_right;
00070     std::string res = passed ? "| chi2 PASS " : "| chi2 FAIL ";
00071     std::cout << res;
00072
00073     return passed;
00074 }

```

2.11 /Users/anastasiatrufanova/Desktop/lab3_generation/src/stats.h File Reference

```

#include <cmath>
#include <iostream>

```

Functions

- long double [mean](#) (const uint32_t *data, int len)
Computes the mean of the given data.
- long double [stddev](#) (const uint32_t *data, int len, long double [mean](#))
Computes the standard deviation of the given data.
- long double [cv](#) (long double [mean](#), long double std_dev)
Computes the coefficient of variation.
- int [checking_chi2](#) (std::vector< uint32_t > &sample)
Checks the chi-squared hypothesis test for the sample.
- double [chi2_teor](#) (int v, double x_p)
Returns theoretical chi-squared value.
- double [chi2_exp](#) (const std::vector< uint32_t > &sample)
Calculates experimental chi-squared value.

2.11.1 Function Documentation

2.11.1.1 [checking_chi2\(\)](#)

```
int checking_chi2 (
    std::vector< uint32_t > & sample )
```

Checks the chi-squared hypothesis test for the sample.

Parameters

<i>sample</i>	Vector of integer sample data.
---------------	--------------------------------

Returns

0 if accepted, 1 if rejected.

Definition at line 60 of file [stats.cpp](#).

2.11.1.2 [chi2_exp\(\)](#)

```
double chi2_exp (
    const std::vector< uint32_t > & sample )
```

Calculates experimental chi-squared value.

Parameters

<i>sample</i>	Vector of integer sample data.
---------------	--------------------------------

Returns

Experimental chi-squared value.

Definition at line 29 of file [stats.cpp](#).

2.11.1.3 chi2_teor()

```
double chi2_teor (
    int v,
    double x_p )
```

Returns theoretical chi-squared value.

Parameters

ν	Degrees of freedom.
χ^2_{1-p}	Significance level (quantile).

Returns

Theoretical chi-squared value.

Definition at line 24 of file [stats.cpp](#).

2.11.1.4 cv()

```
long double cv (
    long double mean,
    long double std_dev )
```

Computes the coefficient of variation.

Parameters

<i>mean</i>	The mean of the data.
<i>std_dev</i>	The standard deviation.

Returns

The coefficient of variation (as a percentage).

Definition at line 18 of file [stats.cpp](#).

2.11.1.5 mean()

```
long double mean (
    const uint32_t * data,
    int len )
```

Computes the mean of the given data.

Parameters

<i>data</i>	Pointer to the array of values.
<i>len</i>	Number of elements in the array.

Returns

The mean value.

Definition at line 2 of file [stats.cpp](#).

2.11.1.6 stddev()

```
long double stddev (
    const uint32_t * data,
    int len,
    long double mean )
```

Computes the standard deviation of the given data.

Parameters

<i>data</i>	Pointer to the array of values.
<i>len</i>	Number of elements in the array.
<i>mean</i>	The mean of the data.

Returns

The standard deviation.

Definition at line 9 of file [stats.cpp](#).

2.12 stats.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STATS_H
00002 #define STATS_H
00003 #include <cmath>
00004 #include <iostream>
00011 long double mean(const uint32_t* data, int len);
```

```
00012
00020 long double stddev(const uint32_t* data, int len, long double mean);
00021
00028 long double cv(long double mean, long double std_dev);
00029
00030
00036 int checking_chi2(std::vector<uint32_t> &sample);
00037
00044 double chi2_teor(int v, double x_p);
00045
00051 double chi2_exp(const std::vector<uint32_t>& sample);
00052
00053 #endif
```

Index

/Users/anastasiatrufanova/Desktop/lab3_generation/src/gen.cpp, mid_square_sample, 12
3, 7
mt_sample, 12
/Users/anastasiatrufanova/Desktop/lab3_generation/src/gen.h, time, 13
10, 14
time_seed, 13
/Users/anastasiatrufanova/Desktop/lab3_generation/src/nist.cpp, xorshift_next, 13
14, 17
xorshift_sample, 13
/Users/anastasiatrufanova/Desktop/lab3_generation/src/nist.h, generate_and_analyze
19, 22
gen.cpp, 4
/Users/anastasiatrufanova/Desktop/lab3_generation/src/stats.cpp, gen.h, 10
22, 25
/Users/anastasiatrufanova/Desktop/lab3_generation/src/stats.h, lcg_next
26, 29
gen.cpp, 4
gen.h, 11
lcg_sample
gen.cpp, 4
gen.h, 11
longest_run_test
nist.cpp, 16
nist.h, 21
main
gen.cpp, 5
mean
stats.cpp, 24
stats.h, 28
mid_square_next
gen.cpp, 5
gen.h, 11
mid_square_sample
gen.cpp, 5
gen.h, 12
monobit_test
nist.cpp, 16
nist.h, 21
mt_sample
gen.cpp, 6
gen.h, 12
nist.cpp
block_frequency_test, 15
cusum_test, 15
longest_run_test, 16
monobit_test, 16
Phi, 16
runs_test, 17
nist.h
block_frequency_test, 20
cusum_test, 20
longest_run_test, 21
monobit_test, 21

block_frequency_test
nist.cpp, 15
nist.h, 20

checking_chi2
stats.cpp, 23
stats.h, 27

chi2_exp
stats.cpp, 23
stats.h, 27

chi2_teor
stats.cpp, 24
stats.h, 28

cusum_test
nist.cpp, 15
nist.h, 20

cv
stats.cpp, 24
stats.h, 28

gen.cpp
generate_and_analyze, 4
lcg_next, 4
lcg_sample, 4
main, 5
mid_square_next, 5
mid_square_sample, 5
mt_sample, 6
time, 6
time_seed, 6
xorshift_next, 6
xorshift_sample, 7

gen.h
generate_and_analyze, 10
lcg_next, 11
lcg_sample, 11
mid_square_next, 11

- Phi, [21](#)
 - runs_test, [22](#)
- Phi
 - nist.cpp, [16](#)
 - nist.h, [21](#)
- runs_test
 - nist.cpp, [17](#)
 - nist.h, [22](#)
- stats.cpp
 - checking_chi2, [23](#)
 - chi2_exp, [23](#)
 - chi2_teor, [24](#)
 - cv, [24](#)
 - mean, [24](#)
 - stddev, [25](#)
- stats.h
 - checking_chi2, [27](#)
 - chi2_exp, [27](#)
 - chi2_teor, [28](#)
 - cv, [28](#)
 - mean, [28](#)
 - stddev, [29](#)
- stddev
 - stats.cpp, [25](#)
 - stats.h, [29](#)
- time
 - gen.cpp, [6](#)
 - gen.h, [13](#)
- time_seed
 - gen.cpp, [6](#)
 - gen.h, [13](#)
- xorshift_next
 - gen.cpp, [6](#)
 - gen.h, [13](#)
- xorshift_sample
 - gen.cpp, [7](#)
 - gen.h, [13](#)